Haven TRD

Oct 19, 2026 · @Athar Ali

Tech stack

Everything is TypeScript. Supabase handles database, login, files, and realtime so we write less backend code.

Layer	Choice	Why
Repo	Monorepo (pnpm workspaces)	One place for all apps and shared types
Citizen app	React + Vite PWA	Mobile-friendly first; React Native/Expo later
Admin dashboard	React + Vite	Separate app, same repo
UI	Tailwind + shadcn/ui, Leaflet + OpenStreetMap	Fast UI, free maps
Backend	Node.js + Express (or Fastify), Zod	Simple, typed validation
Database	Supabase Postgres + PostGIS + pgvector	Location queries and legal search
Auth / OTP	Supabase Auth phone OTP	Twilio or a local SMS provider
Storage	Supabase Storage (private buckets)	Audio and photos stay private
Realtime	Supabase Realtime	Easier than raw WebSockets
AI	Gemini API	Speech-to-text, summary, category, urgency, embeddings
PDFs	pdf-lib or Puppeteer	Generate documents
Hosting	Vercel (frontends), Render/Railway (API), Supabase (DB)	Express needs a long-running server, which Vercel serverless handles poorly

Repo and environments

One monorepo called `haven`, with three apps and one shared package.

```
haven/
  apps/
    citizen/   React + Vite PWA
    admin/     React + Vite dashboard
    api/       Node.js + Express + TypeScript
  packages/
    shared/    types and Zod schemas
  docs/        project documents
```

- Root files: README, `.gitignore`, `.env.example`.
- Code quality: ESLint, Prettier, and Husky hooks.
- Branches: `main`, `dev`, and feature branches.
- CI: a basic GitHub Actions lint and build check.
- Environments: dev and prod use separate Supabase projects.

Database

Supabase Postgres with PostGIS (locations) and pgvector (legal search). Enums are used for urgency, status, category, and role.

Table	Holds
users	Phone, role, language, trust score
complaints	Text, audio, summary, category, AI urgency, final urgency, location, anonymous flag, status
attachments	Photos and audio linked to a complaint
playbooks	Safety steps per scenario: right now, next hour, next 24 hours
legal_entries	Law name, section, plain summary, source link, embedding
documents	Generated drafts and their PDFs
teams	Response teams and departments, area, contact, availability
assignments	Team assigned to a case, with ETA
status_updates	Status history shown to the user
admin_actions	Accept, reject, urgency change, each with a reason

Table	Holds
audit_logs	Every view and change: who, what, when

Row Level Security (per role)

- Citizen sees only their own cases.
- Control room sees safety cases.
- Department officer sees civic cases only.
- SuperAdmin manages users, content, and audit logs.
- Anonymous identity is visible only to authorized senior admins.

Storage and seed data

- Private buckets for audio and photos. A retention rule deletes audio after the case closes and the retention period ends.
- Seed: 3 playbooks (harassment, theft, one civic), the first batch of legal entries, and sample teams.

Backend API

Node.js + Express (or Fastify) in TypeScript. Every request is checked with Zod schemas shared from `packages/shared`.

Area	What it does
Auth	Phone OTP through Supabase Auth. Middleware checks the token and the role.
Complaints	Create, update, and change status.
AI pipeline	Runs when a complaint is created (next section).
Admin actions	Accept, reject, change urgency, assign team with ETA. Each one writes an audit log.
Duplicate check	Same text, same place, same time window.
Rate limits	A maximum number of complaints per user and per device per day.
Realtime	Status updates pushed through Supabase Realtime.
SMS	Status messages through the SMS provider.

Role guards follow the same rules as the database: citizens see their own cases, the control room sees safety cases, officers see civic cases only.

AI pipeline

Gemini does the AI work. Our own rules sit around it so the AI can never lower a danger level.

1. **Danger keyword rules** run first (threats, weapons, "following me", "help"). They can only raise urgency.
2. **Speech to text** for voice. The user confirms or edits the text.
3. **Translation** to a working language. The original text is always kept.
4. **Summary, category, and urgency** with reasons.
5. **Final urgency** is the higher of the rules and the AI. An admin can override it, and the override is logged.
6. **Playbook selector** picks approved safety steps for the scenario.

Safety rules

- The AI never lowers urgency when danger words are present.
- If the AI is unsure, the case is marked for human review and shows safe default steps.
- If the AI fails, the case becomes High with default steps, and emergency buttons stay visible.
- Use AI settings with no training and limited data retention. Do not keep raw data with the provider longer than needed.

Legal content and documents

We curate all legal and playbook content ourselves from official law text. There is no lawyer review step. A SuperAdmin manages it in the admin panel (content manager).

- Each legal entry stores the law name, section, a plain summary, and a source link.
- The rights explainer searches only these entries (pgvector). It explains from them and does not guess laws.
- Every answer shows the law and section. Every legal screen shows: "General guidance, not a substitute for a lawyer."
- Starting laws: Pakistan Penal Code, Code of Criminal Procedure, Prevention of Electronic Crimes Act 2016, Protection Against Harassment of Women at the Workplace Act 2010, and the Sindh Domestic Violence (Prevention and Protection) Act 2013.

- Check each section and any amendment against the official text before it goes in, and update it when the law changes.

Documents (PDF with pdf-lib or Puppeteer)

Document	When	Stage
Complaint application	Every safety case	MVP
Evidence checklist	All cases	MVP
Civic application	Civic cases	MVP
FIR draft (offences marked "for officer confirmation")	High and Critical criminal cases	Later